Quadrotor Simulation in MATLAB: Nonlinear and Linearized Models

System Analysis, Numerical Linearization, Stability, State Feedback and Observer Design

Prepared by: Amirali Bagheral

Instructor: Dr. Nouri Manzar

Course Project — Control Systems

Technical report based on the project reports and submitted MATLAB source files

Abstract

This project develops and studies a twelve-state nonlinear quadrotor model in MATLAB, derives local linear state-space representations around selected operating conditions, and uses the resulting models to investigate stability, controllability, observability, state-feedback control, and state estimation. The workflow combines numerical integration of the nonlinear equations, finite-difference Jacobian calculations, nonlinear-versus-linearized response comparisons, eigenvalue analysis, pole placement, linear-quadratic regulator (LQR) design, Luenberger observer construction, and integral-action control. The MATLAB scripts organize these tasks into reusable functions, with main.m coordinating the simulation sequence. The work establishes a coherent model-to-control workflow and produces diagnostic plots and numerical checks. The report also distinguishes implemented calculations from conclusions that require further numerical validation; several implementation details that affect interpretation are listed in the verification notes near the end.

Keywords: quadrotor, nonlinear dynamics, state-space model, Jacobian linearization, MATLAB, stability, controllability, observability, pole placement, LQR, observer, integral control.

Project Scope and Workflow

The project is organized as a sequence of modeling and control tasks. First, a nonlinear rigid-body model is implemented and simulated. Physical input, state, and output bounds are then defined, and hover and vertical-motion operating conditions are considered. The model is linearized numerically at hover; the nonlinear and linearized models are compared under several thrust perturbations. The local model is subsequently examined using eigenvalues and controllability/observability tests. Controller design progresses from state-feedback methods to observer-based estimation and integral action.

Stage	MATLAB source	Purpose
Model and operating conditions	system_model.m	Parameters, nonlinear dynamics, constraints, operating-point data, initial plots.
Numerical linearization	system_jacobian.m	Central finite-difference approximations of A and B.
Model validation	linearization_check.m	Compare nonlinear and linearized trajectories for several input-step sizes.
Stability and structural tests	task6_7_stability_controllability.m	Eigenvalues, time constants, pole plot, controllability and observability ranks.
Feedback design	controller_design.m	Pole-placement feedback, LQR comparison, simulated response and performance metrics.
State estimation	observer_design.m	Luenberger observer gain, observer poles and true-versus-estimated state plots.
Integrated control tests	combined_simulation.m	State feedback with observer, integral-action augmentation, and combined observer/integral structure.
Execution sequence	main.m	Calls the model, linearization, validation, analysis, controller, observer and combined routines.

1. Nonlinear System Analysis and Modeling (Tasks 1–3)

Task 1: Nonlinear Modeling and Dynamic Simulation

The model represents a rigid quadrotor with three-dimensional translation and three-dimensional attitude. Its state is composed of position, Euler angles, translational velocity, and body angular rates. In the submitted implementation, the state ordering is important because later scripts access individual states by their index.

The state vector used by the code is:

The input vector contains total thrust and three control moments/torques. The nonlinear model uses the mass, gravitational acceleration, arm length, and principal moments of inertia as parameters. The implemented parameter values are m = 1 kg, g = 9.81 m/s^2, L = 0.2 m, Ix = Iy = 0.02 kg·m^2, and Iz = 0.04 kg·m^2. These are simulation parameters in the code, not a claim that they describe a particular commercial aircraft.

Translational accelerations are obtained by resolving thrust through the attitude angles. The expressions contain products and trigonometric functions of roll, pitch, and yaw, so the model is nonlinear and couples attitude to translation. Rotational accelerations are based on Euler rigid-body equations and include gyroscopic cross-coupling terms involving the angular rates and inertia differences.

The equations are integrated with MATLAB ode45. In the initial demonstration, the model starts from zero states and uses thrust slightly above the weight, together with a small control-moment perturbation. The source code then plots altitude, roll angle, and yaw angle. These plots are intended to make the response visible rather than leave the simulation at an all-zero equilibrium.

$$x = [x, y, z, \varphi, \theta, \psi, v_x, v_y, v_z, p, q, r]^T \in \mathbb{R}^{12}$$

$$u = [u_1, u_2, u_3, u_4]^T$$

$$\dot{x} = f(x,u)$$

Task 2: Input, State, and Output Constraints

The code defines admissible ranges for the four control inputs, six outputs, and twelve states. The thrust input is constrained to be non-negative and no greater than twice the vehicle weight. The three moment inputs are assigned finite positive and negative limits. Position, attitude, velocity, and angular-rate limits are also encoded in the model structure.

These bounds describe the intended operating envelope and provide context for interpreting simulations. They do not, by themselves, enforce saturation in every subsequent controller simulation: the controller-design functions use linear state-space models and do not consistently apply actuator saturation. Accordingly, the bounds should be understood as declared constraints unless a simulation explicitly applies them.

$$0 \le u_1 \le 2mg$$

$$-0.2 \le u_2 \le 0.2;\ -0.2 \le u_3 \le 0.2;\ -0.1 \le u_4 \le 0.1$$

Task 3: Operating Points and Equilibrium Checks

Two conditions are stored in the operating_points structure. The hover point uses zero state values and thrust equal to the weight, with all three moment inputs equal to zero. This is the nominal equilibrium used by the Jacobian routine and subsequent linear analysis.

The code also labels a condition “climb,” assigning a vertical velocity of 1 m/s while retaining the hover thrust. This is a constant-velocity climbing trajectory in the simplified translational equations, rather than a stationary equilibrium in the full twelve-state representation: the altitude derivative is nonzero when v_z = 1. This distinction matters when interpreting the printed derivative check and the linearization option.

A phase-portrait vector field in the (z, v_z) plane is also generated around hover. The script uses the vertical kinematic relation ż = v_z and a vertical acceleration expression based on gravity and thrust. The phase portrait is a qualitative view of local vertical motion, not a substitute for the full twelve-state dynamics.

$$\textit{Hover: } x_e = 0_{12},\quad u_e = [mg, 0, 0, 0]^T$$

$$\textit{General local equilibrium condition: } f(x_e,u_e) = 0$$

2. Numerical Linearization and Model Comparison (Tasks 4–5)

Task 4: Jacobian-Based Linearization

The function system_jacobian(SYS, op) selects the stored hover or climb operating condition and estimates the local state and input Jacobians by central finite differences. A small perturbation $\varepsilon = 10^{-6}$ is applied independently to each state and each input. Central differences are used because they generally provide a more accurate numerical derivative than a one-sided difference for a smooth function, at the cost of evaluating the nonlinear model at both positive and negative perturbations.

The resulting A matrix describes how the state derivatives change locally with the states, while B describes their local sensitivity to the inputs. The linear approximation is expressed in deviation variables about the selected nominal condition. It is only expected to approximate the nonlinear system well in a neighborhood of that point; accuracy usually degrades as the state or input deviations become larger.

$$A_{ij} \approx [f_i(x_e + \varepsilon e_j, u_e) - f_i(x_e - \varepsilon e_j, u_e)]/(2\varepsilon)$$

$$B_{ij} \approx [f_i(x_e, u_e + \varepsilon e_j) - f_i(x_e, u_e - \varepsilon e_j)]/(2\varepsilon)$$

$$\delta\dot{x} = A\delta x + B\delta u, \quad where\ \delta x = x - x_e\ and\ \delta u = u - u_e$$

Task 5: Nonlinear-versus-Linearized Response Validation

linearization_check.m computes A and B around hover, initializes the nonlinear simulation with nonzero altitude, roll, and horizontal-velocity deviations, and applies three thrust perturbations corresponding to 5%, 10%, and 20% of mg. For each case, the nonlinear differential equations are integrated with ode45. The linear model is simulated using the same input deviation and an initial state expressed relative to the operating point; its state deviations are then shifted back to absolute coordinates for comparison.

The scripts compare altitude z, roll φ, horizontal velocity v_x, and vertical velocity v_z. The nonlinear response is interpolated onto the fixed linear simulation time grid so that pointwise differences can be calculated. Absolute errors and per-state root-mean-square errors (RMSE) are computed, and plots show the nonlinear trajectory, linearized trajectory, and absolute error.

This experiment is designed to test the local validity of the Jacobian model and to show how approximation error changes with perturbation magnitude. The submitted code prints the RMSE values, but the source files alone do not establish a particular numerical accuracy level; any claim about the size or trend of the errors should be based on the actual MATLAB run output.

$$e_i(t) = |x_{nl,i}(t) - x_{lin,i}(t)|$$

$$RMSE_i = \sqrt{[(1/N)\,\Sigma_k\,(x_{nl,i}(t_k) - x_{lin,i}(t_k))^2]}$$

3. Stability, Controllability, and Observability (Tasks 6–7)

Task 6: Open-Loop Stability Analysis

task6_7_stability_controllability.m calculates the eigenvalues of the hover linearization matrix A. The real parts of these eigenvalues are used to classify the linearized open-loop system as asymptotically stable, unstable, or marginally stable. The script also computes modal time constants from $-1/\mathrm{Re}(\lambda)$ and identifies the eigenvalue with the largest real part as the dominant pole.

A pole plot displays eigenvalues in the complex plane. A small thrust step is also applied to the linearized open-loop system to illustrate the unregulated response. The stability classification should be interpreted from the actual computed eigenvalues; the code's printed interpretation that a quadrotor is generally unstable or marginally stable is a qualitative statement, not a substitute for the eigenvalue test. Modes with zero real part require care because the simple time-constant expression is infinite or undefined for such modes.

$$\text{Characteristic equation: } \det(\lambda I - A) = 0$$

$$\tau_i = -1/\mathrm{Re}(\lambda_i), \text{ for modes with } \mathrm{Re}(\lambda_i) \neq 0$$

Task 7: Controllability and Observability

The controllability matrix is constructed using MATLAB ctrb(A,B). Its rank is compared with the state dimension n = 12. Full rank means that, for the linear time-invariant model, every state can be reached from the origin in finite time under suitable inputs. If the rank is lower than n, at least one state direction is uncontrollable in this linear model.

For observability, the code assumes full-state output measurement by setting $C = I_n$ and constructs obsv(A,C). Under this idealized measurement assumption, the full state is directly available as the output, so full observability is expected mathematically. This is a structural test of the selected output model; it does not demonstrate that all twelve states can be measured directly on a real quadrotor with practical sensors.

$$\mathcal{C} = [B, AB, A^2B, \ldots, A^{n-1}B]$$

$$\mathcal{O} = [C^T, (CA)^T, (CA^2)^T, \ldots, (CA^{n-1})^T]^T$$

$$\text{Full controllability/observability: } \mathrm{rank}(\mathcal{C}) = n \text{ and } \mathrm{rank}(\mathcal{O}) = n$$

4. State-Feedback Controller Design (Tasks 8–10)

Task 8: Pole Placement and LQR Comparison

controller_design.m implements two feedback-design approaches. In the pole-placement method, the desired dominant poles are selected using a damping ratio $\zeta = 0.7$ and a nominal settling-time parameter $T_s = 2$ s. The code calculates a corresponding natural frequency, creates a complex-conjugate pole pair, and adds distinct real poles for the remaining state dimensions. MATLAB place then computes the feedback matrix K so that the eigenvalues of A − BK match the desired locations, subject to controllability and numerical feasibility.

The second method uses a linear-quadratic regulator. The cost matrices are Q = I and R = I, giving equal unit weights to the state-error and input-effort terms in the chosen coordinates. MATLAB lqr returns the gain and closed-loop poles. The code plots the pole-placement and LQR poles together, allowing a qualitative comparison of closed-loop eigenvalue locations. Because Q and R are identity matrices, the design does not explicitly normalize the different physical units of position, angle, velocity, and control input; a scaled weighting design may be more meaningful for a physical implementation.

$$u = -Kx$$

$$J = \int_0^\infty (x^T Q x + u^T R u)\, dt$$

Task 9: Closed-Loop Simulation and Tracking Metrics

The controller script forms A_cl = A − BK and simulates a ten-second response with a step reference. It initializes the system with an altitude-state offset and a five-degree roll offset, and calculates settling time, overshoot, rise time, and final tracking error from the simulated output. A prefilter is included to scale the reference input in an attempt to obtain unit steady-state gain for the selected output channel.

There is an important indexing detail in the submitted implementation: the state vector places altitude z at index 3, but the Task 9 code selects C(1,:) as the tracked output and comments that this is the z-position. Therefore, as written, the plotted output is the first state x rather than altitude z. The reference prefilter and reported metrics consequently apply to that first state unless the output selector is corrected to use index 3 and the corresponding input-to-output gain is re-derived.

$$A_cl = A - BK$$

$$e_ss = |r(t_f) - y(t_f)|$$

Task 10: Interpretation of the Feedback Design

The design stage demonstrates two standard ways to stabilize a linearized model: direct eigenvalue assignment and optimal feedback based on a quadratic performance index. Pole placement makes the desired transient pole locations explicit, whereas LQR balances state deviations and control effort through Q and R. The submitted plots and response metrics are the appropriate evidence for comparing performance. No numerical performance values are asserted here because they depend on the actual execution results and, in Task 9, on resolving the output-index issue described above.

5. State Estimation and Observer-Based Control (Tasks 11–14)

Task 11: Luenberger Observer Design

observer_design.m assumes $C = I_n$ and places all desired observer poles at −5. The observer gain L is computed by applying pole placement to the dual pair (A^T, C^T), then transposing the result. The eigenvalues of A − LC are displayed to verify the observer error dynamics. In the ideal linear model, the estimation error evolves according to $\dot{e} = (A - LC)e$ when the plant and observer receive the same input and the output model is consistent.

The selected observer poles are intended to make estimation errors decay rapidly. In practical systems, however, placing observer poles very far to the left can amplify measurement noise and sensitivity to modeling error; the pole locations should therefore be assessed against sensor quality and sampling rate.

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x})$$

$$\dot{e} = (A - LC)e, \quad e = x - \hat{x}$$

Task 12: Observer Simulation and Estimation Error

The observer simulation uses a nonzero initial altitude state for the plant and an opposite-sign initial altitude estimate. The plant and observer are assembled into an augmented state-space system, simulated, and separated into true states, estimated states, and estimation error. The first four state channels are plotted individually, with true, estimated, and error traces shown together.

The source uses only the first input channel of the augmented input matrix when constructing the simulation model, while the original quadrotor has four inputs. Thus this observer test exercises the first input channel rather than a general four-input command sequence. The full-state measurement assumption also makes this an idealized observer demonstration.

Task 13: Observer Performance Interpretation

The main qualitative objective is convergence of the estimated state toward the true state despite an initial estimation mismatch. The plots allow convergence and transient error to be inspected. Quantitative claims such as a specific convergence time or estimation accuracy require reading the generated MATLAB plots or calculating metrics from the simulation data; these values are not hard-coded in the report.

Task 14: State Feedback with Observer

combined_simulation.m builds a state-feedback gain using pole placement and constructs an observer gain with poles shifted farther left than the controller poles. The script simulates the closed-loop state and the observer model, then plots true altitude, estimated altitude, and altitude estimation error. This structure demonstrates the separation of two design tasks: choosing a feedback gain for the plant and choosing an observer gain for reconstructing the state.

The source uses C = I for the observer and chooses controller poles spanning approximately −1 to −3. Observer poles are shifted by −2 from those controller poles. The observer is therefore designed to respond faster than the

controller in the nominal linear model, though the actual response and numerical robustness should be checked after execution.

$$u = -K\hat{x}$$

$$\textit{Observer error dynamics: } \dot{e} = (A - LC)e$$

6. Integral Action and Combined Architecture (Tasks 15–17)

Task 15: Integral-Action Formulation

The integrated-control script defines a single-output selector for altitude, C_int, with its nonzero element at state index 3. It augments the original state with an integral state ξ that accumulates the difference between the altitude reference and the measured altitude. This augmentation is intended to remove steady-state error for a constant reference when the augmented system is controllable and the closed-loop design is valid.

$$\xi = r - C_int\, x, \quad with\ C_int\, x = z$$

Task 16: Augmented-State Feedback with Integral Action

The script constructs the augmented matrices by appending the integral state to the original twelve states. It checks controllability of the augmented pair, assigns the original controller poles plus an additional pole at −4, and computes the augmented feedback gain. The gain is separated into a state-feedback part Kx and an integral gain Ki.

This is the standard structure for integral control: the controller uses both the estimated or measured state and the accumulated tracking error. Integral action can compensate for constant disturbances or model mismatch, but it can also cause excessive control effort or windup when actuators saturate. The current simulation structure does not include an explicit anti-windup mechanism or apply the previously declared actuator limits.

$$A_aug = [[A, 0], [-C_int, 0]]$$

$$B_aug = [B^T, 0]^T$$

$$u = -Kx\, x - Ki\, \xi$$

Task 17: Observer Plus Integral Controller

The final combined augmented model contains the physical state x, the estimated state x̂, and the integral state ξ. The reference enters the integral-state equation, while the control law uses the state estimate and integral term. The script simulates the augmented system for a unit-step reference and plots actual altitude, estimated altitude, integral state, and control input.

This final stage brings together the core ideas of the project: the plant is represented by a local linear model, the observer estimates the system state, state feedback shapes the closed-loop dynamics, and integral action improves constant-reference tracking. The source code provides the structure for this combined experiment; the final plots and actual closed-loop numerical results should be retained with the submission as evidence of performance.

$$u = -Kx\, \hat{x} - Ki\, \xi$$

$$\xi = r - z$$

7. MATLAB Software Architecture and Reproducibility

The scripts are modular rather than one long monolithic program. system_model.m defines the parameters, nonlinear dynamics, constraints, and operating-point data. system_jacobian.m computes local matrices A and B. linearization_check.m performs the nonlinear-versus-linear comparison. task6_7_stability_controllability.m runs the eigenvalue and rank tests. controller_design.m designs and compares feedback gains. observer_design.m builds

the observer and evaluates state estimation. combined_simulation.m combines observer-based feedback and integral action. main.m calls these components in sequence.

To reproduce the workflow, place all supplied .m files in the same MATLAB working folder, ensure the required MATLAB toolboxes/functions are available (including Control System Toolbox functions such as ss, lsim, ctrb, obsv, place, lqr, and stepinfo), and run main.m. The model source calls helper functions defined within system_model.m; if any additional helper functions are absent from the submitted folder or from the MATLAB path, MATLAB will stop at the first unresolved function. Save the command-window output and generated figures alongside the report so that all numerical claims can be traced to an actual run.

8. Main Achievements and Technical Outcomes

- Implemented a twelve-state nonlinear rigid-body quadrotor model with translational and rotational dynamics and a four-channel control input.
- Defined model parameters, actuator bounds, state/output bounds, and nominal hover data in a reusable MATLAB structure.
- Implemented numerical Jacobian calculations using central finite differences to obtain local A and B matrices.
- Created a repeatable experiment comparing nonlinear and linearized trajectories under three thrust perturbation magnitudes, with per-state RMSE calculations.
- Automated open-loop eigenvalue analysis and rank-based controllability and observability checks.
- Implemented both pole-placement and LQR state-feedback design and added pole comparison plots.
- Constructed a Luenberger observer and simulation plots for true states, estimates, and estimation error.
- Extended the linear model with an integral state and assembled an observer-based integral-feedback simulation.
- Organized the workflow into modular MATLAB functions controlled by a single main script, improving clarity and repeatability.

These achievements describe functionality present in the supplied source files. They should not be read as proof of a specific tracking accuracy, stability margin, or nonlinear-model fidelity until the code has been executed, the plots and numerical outputs reviewed, and the verification notes below resolved.

9. Verification Notes and Limitations

The following points were identified by reading the submitted MATLAB files. They are included to make the report technically transparent and to help prevent overclaiming results. They do not negate the educational value of the project, but they should be reviewed before treating the simulations as a validated control design.

Item	What the code currently does	Why it matters / recommended check
Climb operating point	Sets v_z = 1 while retaining hover thrust and labels the condition as climb.	This is not a full-state equilibrium because ż = v_z ≠ 0. Treat it as a constant-velocity trajectory or formulate a suitable moving-coordinate/trajectory linearization.
Task 9 output index	Uses C(1,:) although the model state ordering places altitude z at index 3.	The plotted output and performance metrics correspond to x, not z. Use the altitude selector C(3,:) and recompute the reference prefilter consistently if altitude tracking is intended.
Observer input channel	observer_design.m builds the augmented simulation with B_aug(:,1), despite the four-input plant.	The simulation drives only the first input channel. If general four-input excitation is intended, supply a four-column input signal and preserve all input channels.

Full-state output assumption	Observer and observability tests use C = I.	This assumes all twelve states are directly measured. A realistic sensor model should specify which outputs are available and reassess observability.
Input constraints	Limits are defined in system_model.m but controller simulations do not consistently saturate control inputs.	Reported control laws may request values outside the declared bounds. Add saturation and, for integral action, consider anti-windup.
Combined simulation interpretation	The combined model simulates an ideal linear plant/observer structure and plots its outputs.	Retain generated plots and inspect dimensions, input/output definitions, and tracking response in MATLAB before claiming closed-loop performance.
Missing execution evidence	The supplied source files define calculations and plotting routines but do not include a MATLAB run log or exported figures.	No numerical RMSE, settling time, overshoot, or observer convergence value is asserted in this report.

10. Conclusion and Future Work

The project presents an end-to-end educational workflow for quadrotor control, starting from nonlinear dynamics and progressing through local linearization, model comparison, stability and structural analysis, feedback design, state estimation, and integral control. The principal contribution is the integration of these topics into a modular MATLAB implementation that can be extended and experimentally evaluated.

The most important next step is to validate the numerical outputs and correct the implementation details that affect interpretation, particularly the altitude-output index in Task 9 and the status of the climbing operating condition. Further improvements could include realistic sensor-output matrices, actuator saturation, measurement noise, external disturbances, parameter sensitivity, systematic scaling of LQR weights, and a direct comparison of nonlinear closed-loop performance against the linearized closed-loop model. These additions would move the project from a structured control-design exercise toward a more complete simulation study.

Appendix A. State and Input Index Reference

Index	State	Meaning
1	x	Position along inertial x axis
2	y	Position along inertial y axis
3	z	Altitude / vertical position
4	φ	Roll angle
5	θ	Pitch angle
6	ψ	Yaw angle
7	v_x	Velocity along x
8	v_Y	Velocity along y
9	v_z	Vertical velocity
10	p	Body angular rate about x
11	q	Body angular rate about y
12	r	Body angular rate about z

Input	Role in code
$\mathbf{u_1}$	Total thrust
$\mathbf{u_2}$	Control moment associated with roll-axis dynamics
$\mathbf{u_3}$	Control moment associated with pitch-axis dynamics
$\mathbf{u_4}$	Control moment associated with yaw-axis dynamics